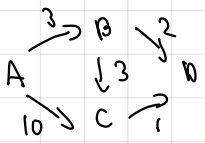
lecture 16
dynamic programming on graphs



Single Source Shortest Path:
starting at node A, what is the shortest path for every other node

Path: sequence of nodes connected by edges
Path weight: sum of edges in path
Shortest Path: weight of path with smallest weight

Shortest paths decompose to other shortest path

Suppose path from u to v
then all of the prefixes are also shortest paths

u → ... w → v
w is the step before v
u → v is also a shortest path

If w is on the way
u → w is also the shortest path

Generally:

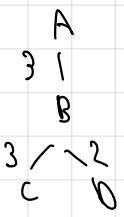
$$\text{shortest_path}(u \to v) = \min_{w} \; \text{shortest_path}(u, w) + w \to v$$

Output: To get the shortest path to a node, what is the next hop
good representation: save space

From A

Node	Pred	Dist
A	—	0
B	A	3
C	B	6
D	B	5

A → B → D
A → B → C



Can make shortest path tree

Recon D → B → A

*in our table, we just need to store the last node in the shortest path

---

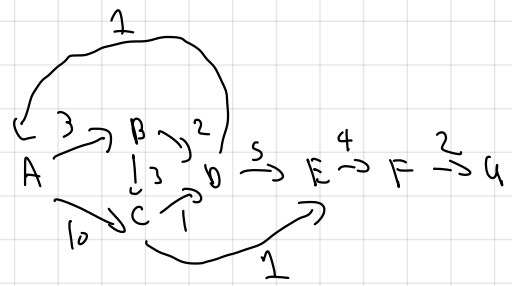
Negative weight cycle

- The path will just be endless loops of the negative weight cycle
- can get around with no duplicate edges or nodes, but that is away from the goal

Assume no negative weight cycles

Node	Pred	Dist
A	—	0
B	A	3
C	B	6
D	B	5
E	C	7
F	E	11
G	F	13

A → B
A → B → C
A → B → D
A → B → C → E
A → B → C → E → F
A → B → C → E → F → G



Relax operation

Both algorithms fill out tables, initially over approximations (∞)
- initially assume no path

Loop through all edges and apply relax operation

```
def relax(u, dst, pred, edge):
    if dst[edge.dst] > dst[edge.src] + edge.weight:
        dst[edge.dst] ← dst[edge.src] + edge.weight
        pred[edge.dst] ← edge.src
```

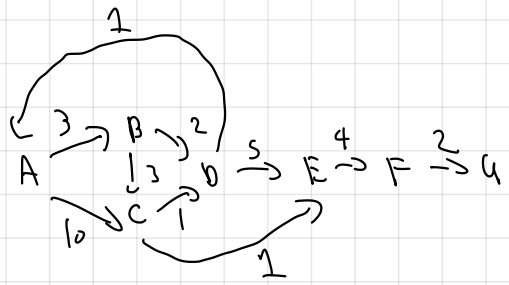
given:
node u
edge $w \xrightarrow{s} v$

On all edges: is this edge the final edge in the path from $u \to v$
starting at node u
given edge $w \to v$ with weight w

$dst[edge.src] + w$ $u \longrightarrow w \to v$	is there a new dist for $u \to v$?

edge $w \to v$
src: w
dst: v

Node	Pred	Dist
A	—	0
B	A	~~∞~~ 3
C	B	~~∞~~ 6
D		∞
E		∞
F		∞
G		∞



relax $A \xrightarrow{3} B$ check if distance to A in table + A→B edge weight 3 is less than distance to B
edge
src: A
dst: B
weight: 3
$0 + 3 < \infty$

B A
$dst[edge.dst] = \infty > dst[edge.src] + edge.weight = 0 + 3$
$dst[edge.dst] = 0 + 3$
$pred[edge.dst] \leftarrow A$

---

relax $B \xrightarrow{3} C$ update $\infty \to 3 + 3$ use this path ✓

relax $A \xrightarrow{10} C$ no update ~~6~~ 10. don't use this path

---

Bellman Ford Algorithm

```
def bellman_ford(graph, start):

    dist = {node: infinity for node in graph}
    dist[start] = 0
    pred = {node: none for node in graph}

    for i = 1, 2, ... n:
        for edge in graph.edges:
            relax(graph, dist, pred, edge)
        if nothing changed:
            break

    return dist, pred

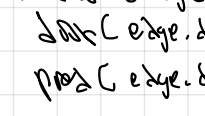
def relax(graph, dist, pred, edge):
    if dist[edge.dst] > dist[edge.src] + edge.weight:
        dist[edge.dst] = dist[edge.src] + edge.weight
        pred[edge.dst] = edge.src
```

setting up: O(n)

loop through edges relax edges

until nothing changes

O(n·m)



node	pred	dist	visited?
A	–	0	✓
B	A	~~∞~~ 1	✓
C	B	~~∞~~ 2	✓
D	A C	~~∞~~ 10 3	✓
E	D	~~∞~~ 4	✓

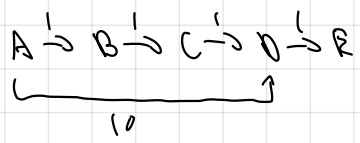
iteration:	relax
1 A	A→B, A→D
2 B	B→C
3 C	C→D
4 D	D→E
5 E	

Dynamically figures out best way to visit nodes
- only visit a node when you have a shorter path to it

There can't be negative weights since the greedy algorithm assumes we have shortest paths to nodes dynamically

recall bellman ford can't have negative cycles

Why n iterations:
- all shortest paths don't have cycles, since those would cost more weights
- only makes sense to have cycles if they are negative weight cycles
  - recall there are no negative cycles
- since there are n nodes, must end in n iterations
- acyclic means < n steps



iterate through all edges

Best:

Node	Pred	Dist
A	–	0
B	A	~~∞~~ 1
C	B	~~∞~~ 2
D	C	~~∞~~ 3
E	D	~~∞~~ 4

A→B	1
B→C	1
C→D	1
D→E	1
A→D	10

Worst:

Node	Pred	Dist
A	–	0
B	A	~~∞~~ 1
C	B	~~∞~~ 2
D	~~A~~ C	~~∞~~ ~~10~~ 3
E	~~D~~ D	~~∞~~ ~~10~~ 4

D→E	1
C→D	1
B→C	1
A→D	10
A→B	1

after 1 iteration, 1 step paths from A
after 2 iterations, 2 step paths from A
⋮
after n iterations, n step paths from A

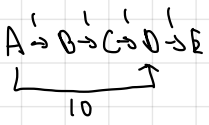
Best
- 2 iterations
- O(n+m)

Worst:
- n iterations
- O(n·m + n)

Dijkstra's Algorithm

Figure out order to relax edges: relax smaller distance

- out of all unvisited nodes, visit the one with the smaller distance
- when you visit a node, relax all outgoing edges



node	pred	dist	visited?
A	–	0	✓
B	A	~~∞~~ 1	✓
C	B	~~∞~~ 2	✓
D	A C	~~∞~~ ~~10~~ 3	✓
E	D	~~∞~~ 4	✓

iteration:	relax
1 A	A→B, A→D
2 B	B→C
3 C	C→D
4 D	D→E
5 E	

```
def dijkstra(graph, start):

    dist = {node: infinity for node in graph}
    dist[start] = 0
    pred = {node: none for node in graph}

    not_visited = set(graph.nodes)          # heap
    while not_visited:                      # O(n) n nodes
        # return element with smallest dist[node] value
        node = not_visited.pop_smallest_dist()   # pop on heap O(log n)
        for edge in graph.Outgoing[node]:        # in total, total O(log n) to update heap
            relax(graph, dist, pred, edge)       # log(n) per edge called
    return dist, pred
```

O(n·log(n) + m·log(n))

O((n+m) log(n))

Unweighted graph:	Weighted graph:
- use a list/queue	- use a min heap
- O(1) insertion	- O(log n) insertion
O(n+m)	O((n+m) log(n))

Breadth first search Dijkstra

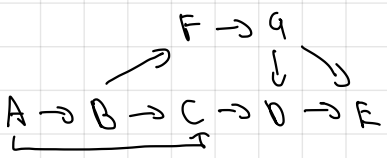
---

run time
- use a heap to find the node of smallest distance. O(log(n)) time search

lecture 17
minimum spanning trees

BFS

Given unweighted graph: equal weight edges



Dijkstra will visit nodes one at a time
- relax all edges on unvisited node with smallest distance [min heap]

Run Dijkstra's Algorithm from node A

node	pred	dist	visited	time dist updated	iteration order	edges
A	-	0	✓	1	A 0	A→B, A→C
B	A	~~∞~~ 1	✓	1	B] 1	B→C, B→F
C	A	~~∞~~ 1	✓	1	C]	C→D
D	C	~~∞~~ 2	✓	1	F] 2	D→E
E	D	~~∞~~ 3	✓	1	D]	
F	B	~~∞~~ 2	✓	1	G] 3	F→G
G	F	~~∞~~ 3	✓	1	E]	G→E, G→D

We visit the nodes in order, first in first out as we relax
- queue/list
- visit nodes in order of distance

When you relax, add nodes in outgoing edge
dist value only gets updated once, like a visited bool

BFS	DFS
layer by layer	go deep

Optimizations

linked list
- appends back O(1)
- pop front O(1)

this linked list can act as a queue
since it has the first in first out property

appends to end
pop from front O(n+m)

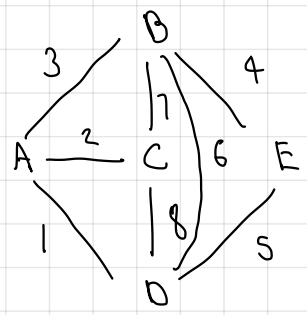
queue/linked list has the elements to visit next

Minimum Spanning Trees

Goal: Connect all nodes together with minimum total edge weights

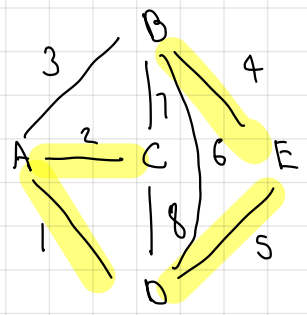
Possible edges

Find MST:



MST: a set of edges that connect all nodes with no cycles and has minimum weight

MST:



total: 10

Spanning Tree

- set of edges that connect all buildings
- removing any edge makes it not connected
- the one with minimum total weight is the MST

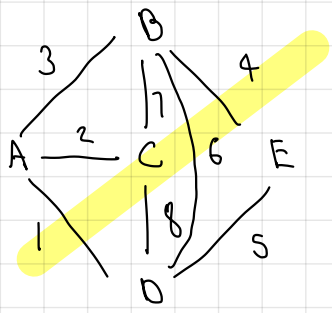
With n nodes, has to have n-1 edges

no such thing as a starting point

Def:
A cut partitions into 2 non-empty subsets
A crossing edge is any edge that starts and ends at different partitions

Lightest crossing edge: lowest weight crossing edge

Example Partitions:
1. {A, C, B}
2. {D, E}



Crossing Edges:
A->D (1)
C->D (8)
B->D (6)
B->E

Lightest Crossing Edge:
A->D

The lightest crossing edge will be an edge in the MST

Assume unique edge weights

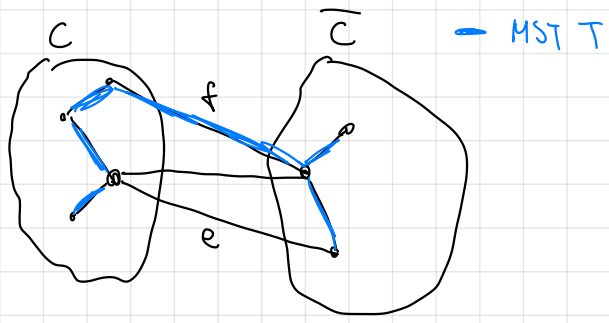
Lemma: Assume g has all distinct edge weights

Let C and $\bar{C}$ be a complement be a cut of the graph
and e be the lightest crossing edge

Then, the lightest crossing edge e is in the MST

Repeatedly apply this to create the MST

Proof by contradiction:
assume e is not in the MST
assume f is in the MST instead

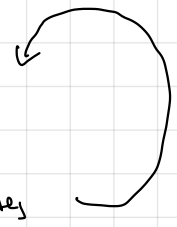


Steps
1. Add e to T to get T' $T' = T \cup \{e\}$, forming a cycle
2. Remove f to remove the cycle $T'' = (T \cup \{e\}) - \{f\}$
3. Weight(T'') < Weight(T)
4. T'' is still a spanning tree since all nodes with f still work with e
   - f was the cross edge that made a cycle with e

If e is not in the MST, there would be a spanning tree with lighter weight (the spanning tree with e). This contradicts the claim that the input spanning tree is a MST.

Greedy Algorithms work on MST

Prim's Algorithm
- starting at any node, build up on blank
- the cut is initially that random node and every other node
- all edge leaving the random node is a crossing edge
- then, the lightest crossing edge must be in the MST, according to the lemma
- now, the cut is all reachable nodes and every other node
- now, the crossing edges are all edges leaving the reachable nodes
- repeat until n-1 edges added



reached is the current cut
outgoing_edges : all crossing edges

```
def prim(graph):

    start_node = (pick any node in graph)

    reached = {start_node}                      current cut so far
    outgoing_edges = graph.out[start_node]      all crossing edges

    MST_edges = set()

    while len(reached) != graph.n_nodes:        n-1 times
        edge = outgoing_edges.pop_lightest()    get lightest crossing edge
        if edge.dst in reached:
            continue                            * min heap pop

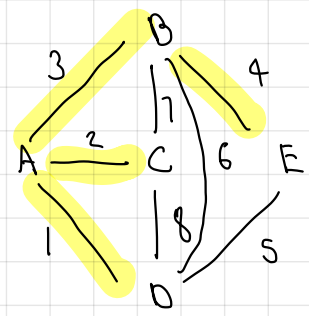
        MST_edges.insert(edge)                  add to MST
        reached.insert(edge.dst)                add to current cut
        for out_edge in graph.out[edge.dst]:    add to candidate crossing edges
            outgoing_edges.insert(out_edge)

    * min heap insert

    return MST_edges
```

Data Structure:
Use a min heap on crossing edges

Given:



Start at node D

Cut	new edge
A, D	A → D
A, D, C	A → C
A, D, C, B	A → B
A, D, C, B, E	B → E

Kruskal's Algorithm
- find the next smallest edge that does not create a cycle

- sort the edges by weight
- iterate through edges and add edges that don't create a cycle
- can have multiple connected components
- Prims Algorithm always has only 1 connected component

```
def kruskal(graph):

    MST_edges = set()
    # sort by weight ascending

    edges = sort(graph.edges)
    for edge in edges:
        src, dst = edge.src, edge.dst
        if not same_component(src, dst)
            MST_edges.insert(edge)
            union(src, dst)
    return MST_edges

def find_root(A)
    if A.parent is empty:
        return A
    A.parent = find_root(A.parent)
    return A.parent

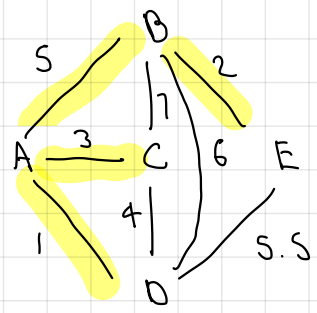
def same_component(A, B):
    return find_root(A) == find_root(B)

def union(A, B):
    A = find_root(A)
    B = find_root(B)

    if A == B:
        return
    if B.size < A.size:
        A, B = B, A

    # now, A is smaller
    A.parent = B
    B.size = B.size + A.size
    return B
```

Given:



edges	weight
AD	1
BE	2
AC	3
CD	4
AB	5
DE	5.5
BD	6
BC	7

MST		
AD	1	
BE	2	
AC	3	
AB	5	can't be CD (no cycles)

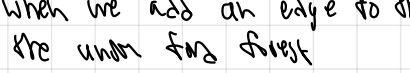
5 nodes → 4 edges in MST

O(n log(n))

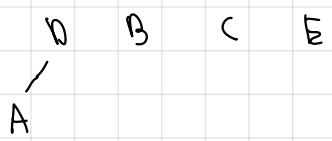
Data Structure
- use union find to quickly determine if a cycle has been introduced

Union Find Forest
- another graph structure with all nodes
- when we add an edge to the MST, connect those nodes in the union find forest

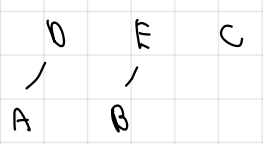
Union Find Forest



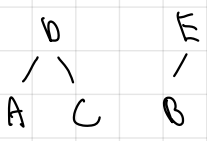
1. Add AD to MST.
   - make A the child of D



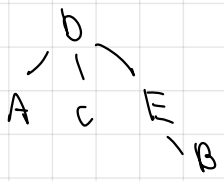
2. Add BE to MST
   - make B the child of E



3. Add AC
   - make C a child of D



4. Add AB

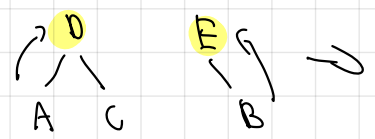


We considered adding CD, but it caused a cycle
- look at the union find parent of both nodes
- C and D both have union find parent D
- if they have the same root in the union find forest, they are already in the same connected
- only add if endpoints are in different connected components

Union

add AB
- find parent of each endpoint

A's parent: D
B's parent: E



if they have the same parent, don't add this edge since it will cause a cycle

1. Make the tree with more nodes the root, keeps the tree small
2. Path compression: A.parent = find_root(A.parent)

